

Bellman-Ford Algorithm

Negative Weights and Cycles

- ⌘ Negative edges are OK, as long as there are no *negative weight cycles* (otherwise paths with arbitrary small “lengths” would be possible).
- ⌘ Shortest-paths can have no cycles (otherwise we could improve them by removing cycles).
 - ⌘ Any shortest-path in graph G can be no longer than $n-1$ edges, where n is the number of vertices

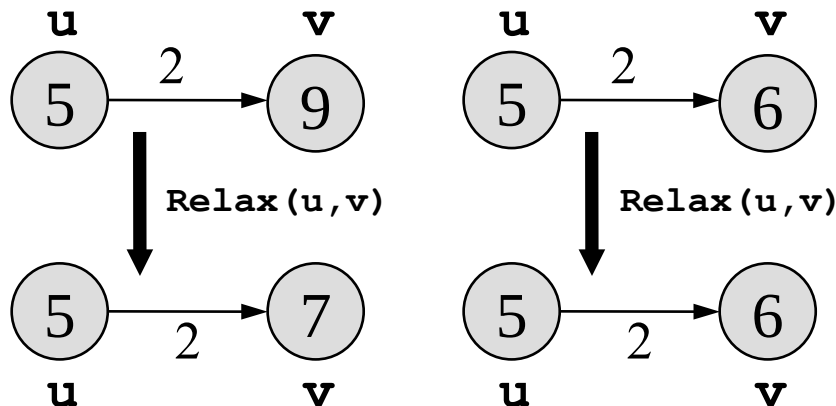
Shortest Path Tree

The result of the algorithms is a *shortest path tree*.
For each vertex v , it

- # records a shortest path from the start vertex s to v .
- # $v.\text{pred}$ is the predecessor of v in this shortest path
- # $v.\text{dist}$ is the shortest path length from s to v

Relaxation

- For each vertex v in the graph, we maintain $v.\text{dist}$, the estimate of the shortest path from s . It is initialized to ∞ at the start.
- Relaxing an edge (u, v) means testing whether we can improve the shortest path to v found so far by going through u .



```
Relax ( $u, v, G$ )  
if  $v.\text{dist} > u.\text{dist} + w(u, v)$  then  
     $v.\text{dist} := u.\text{dist} + w(u, v)$   
     $v.\text{pred} := u$ 
```

Bellman-Ford Algorithm

Dijkstra's doesn't work when there are ***negative edges***:

⌘ Intuition – we cannot be greedy anymore on the assumption that the lengths of paths will only increase in the future

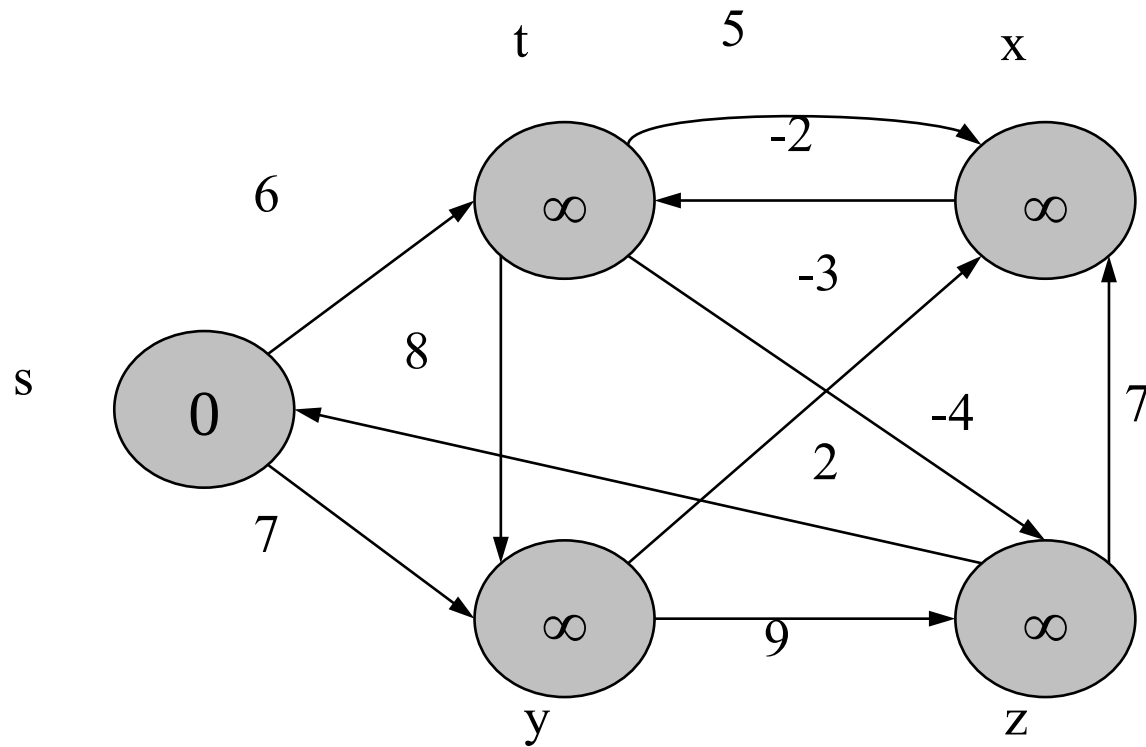
Bellman-Ford algorithm detects negative cycles (returns *false*) or returns the shortest path-tree

Bellman-Ford Algorithm

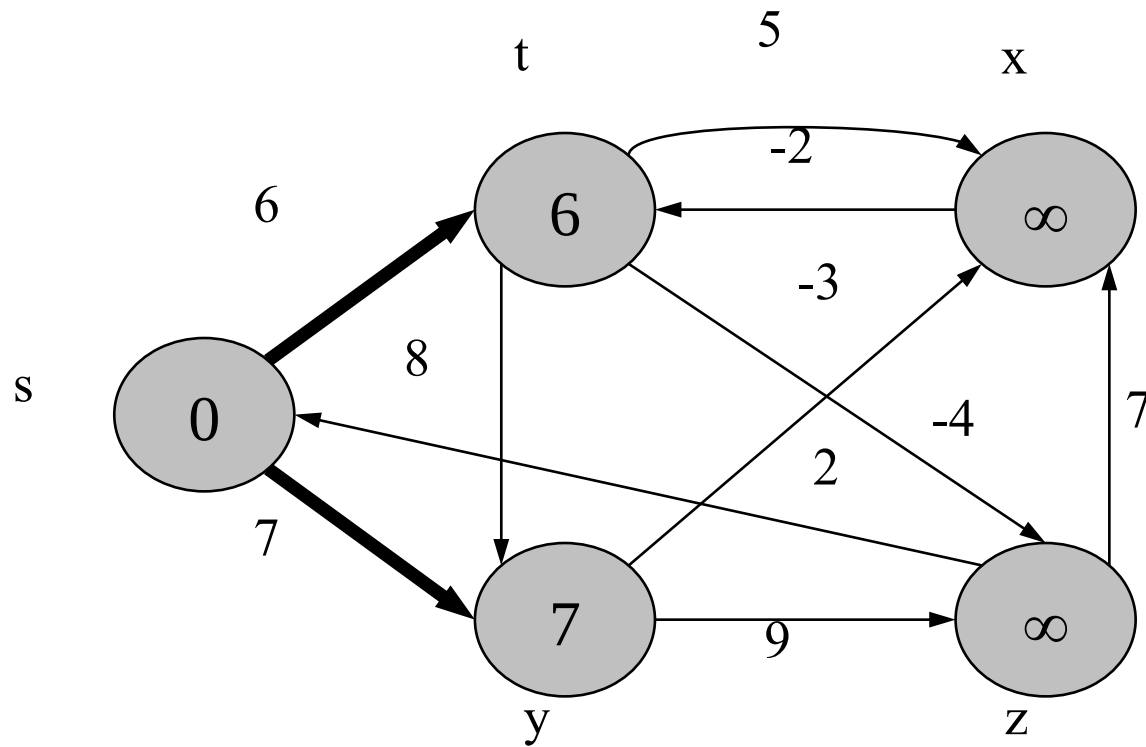
Bellman-Ford(G, s)

```
01 for each vertex  $u \in G.V$ 
02      $u.dist := \infty$ 
03      $u.pred := NIL$ 
04  $s.dist := 0$ 
05 for  $i := 1$  to  $|G.V()|-1$  do
06     for each edge  $(u, v) \in G.E$  do
07         Relax  $(u, v, G)$ 
08 for each edge  $(u, v) \in G.E$  do
09     if  $v.dist > u.dist + w(u, v)$  then
10         return false
11 return true
```

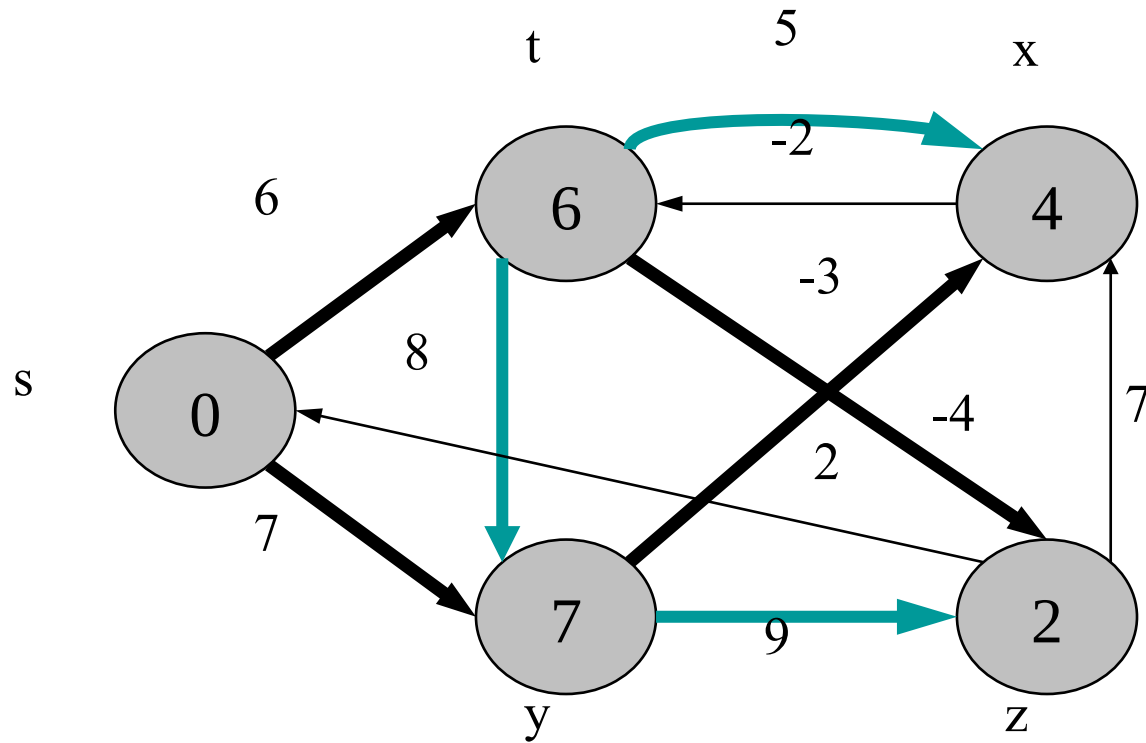
Bellman-Ford Example



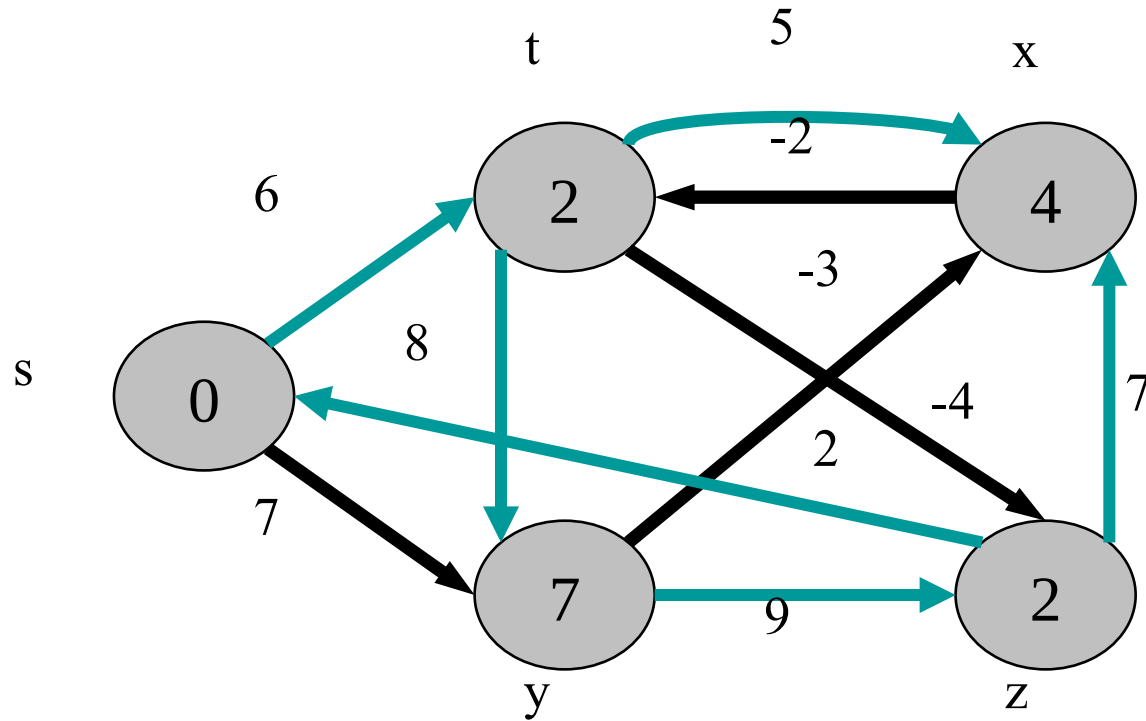
Bellman-Ford Example



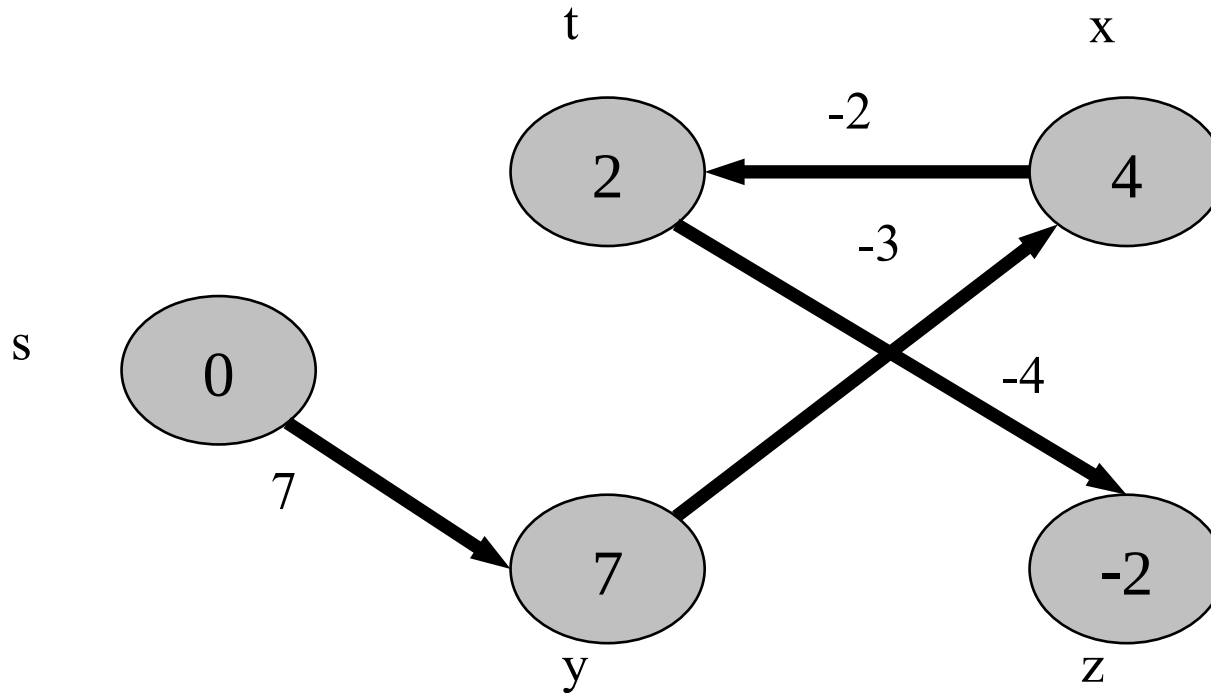
Bellman-Ford Example



Bellman-Ford Example



Bellman-Ford Example



Bellman-Ford Running Time

Bellman-Ford is implemented with nested loops that relax every edge $|E|$ exactly $|V|-1$ times.

✚ The algorithm is based on the principle that the shortest simple path between any two vertices can have at most $|V|-1$ edges.

Bellman-Ford Running Time

Negative Cycle Check (Final Pass): $O(|E|)$

✚ A final, single pass over all $|E|$ edges is performed. If any distance can be further improved (relaxed) in this final pass, it proves that a negative cycle exists and is reachable from the source.

Overall running time: $O((|V|-1)|E|) + O|E| = O(|V|.|E|)$